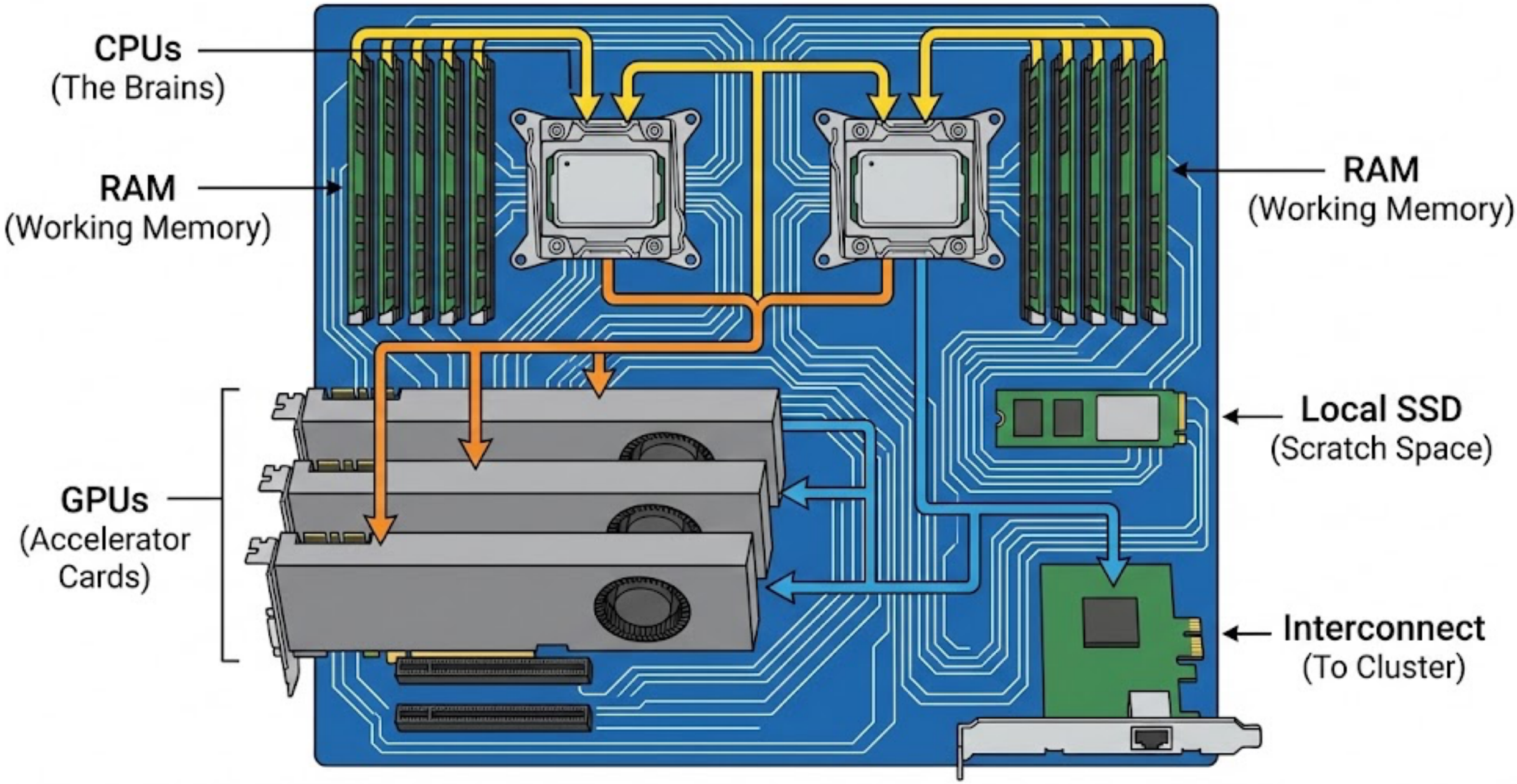
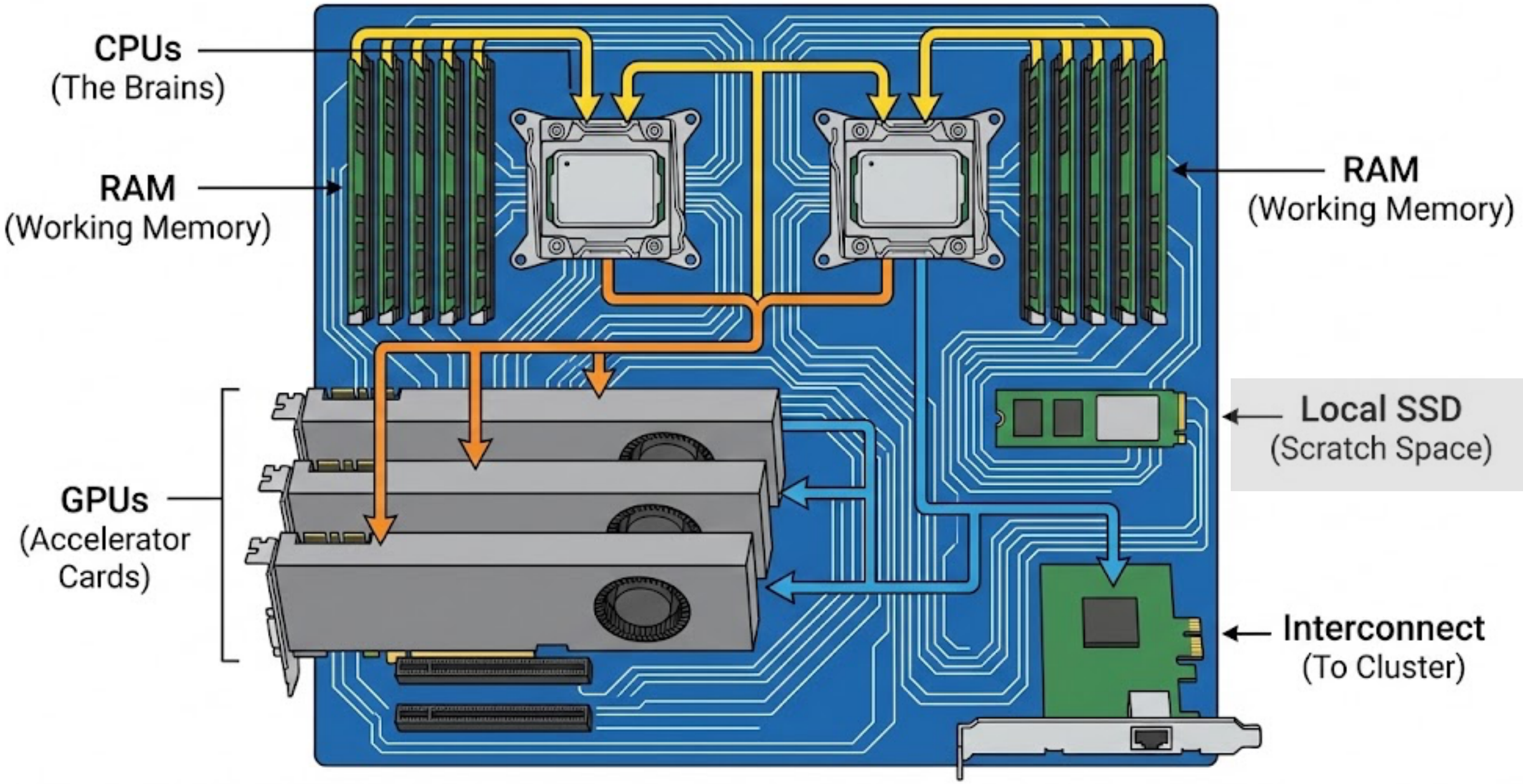
Lecture 12: GPU computing in Python

CEE 690

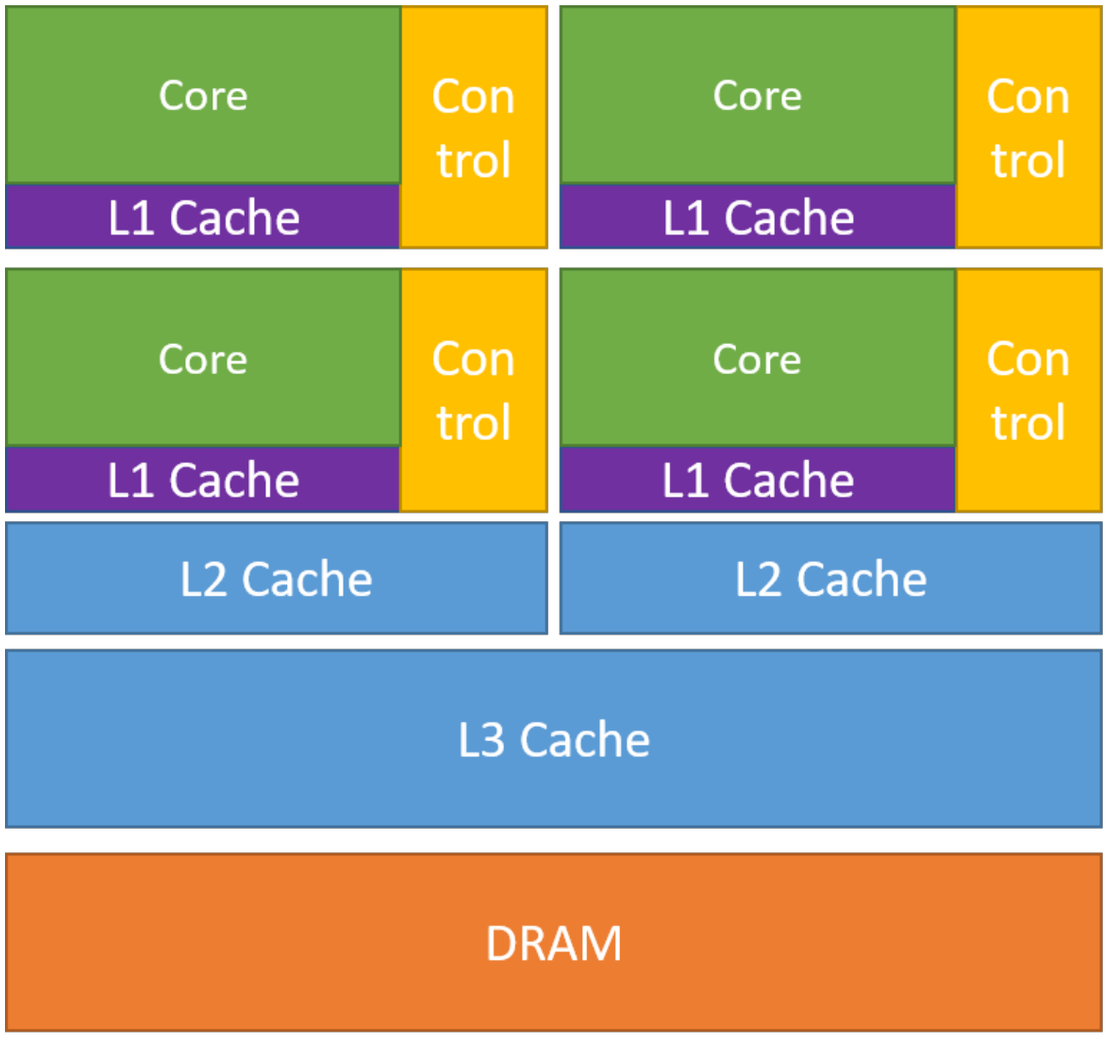
HPC Compute Node Schematic - Anatomy of a Workhorse



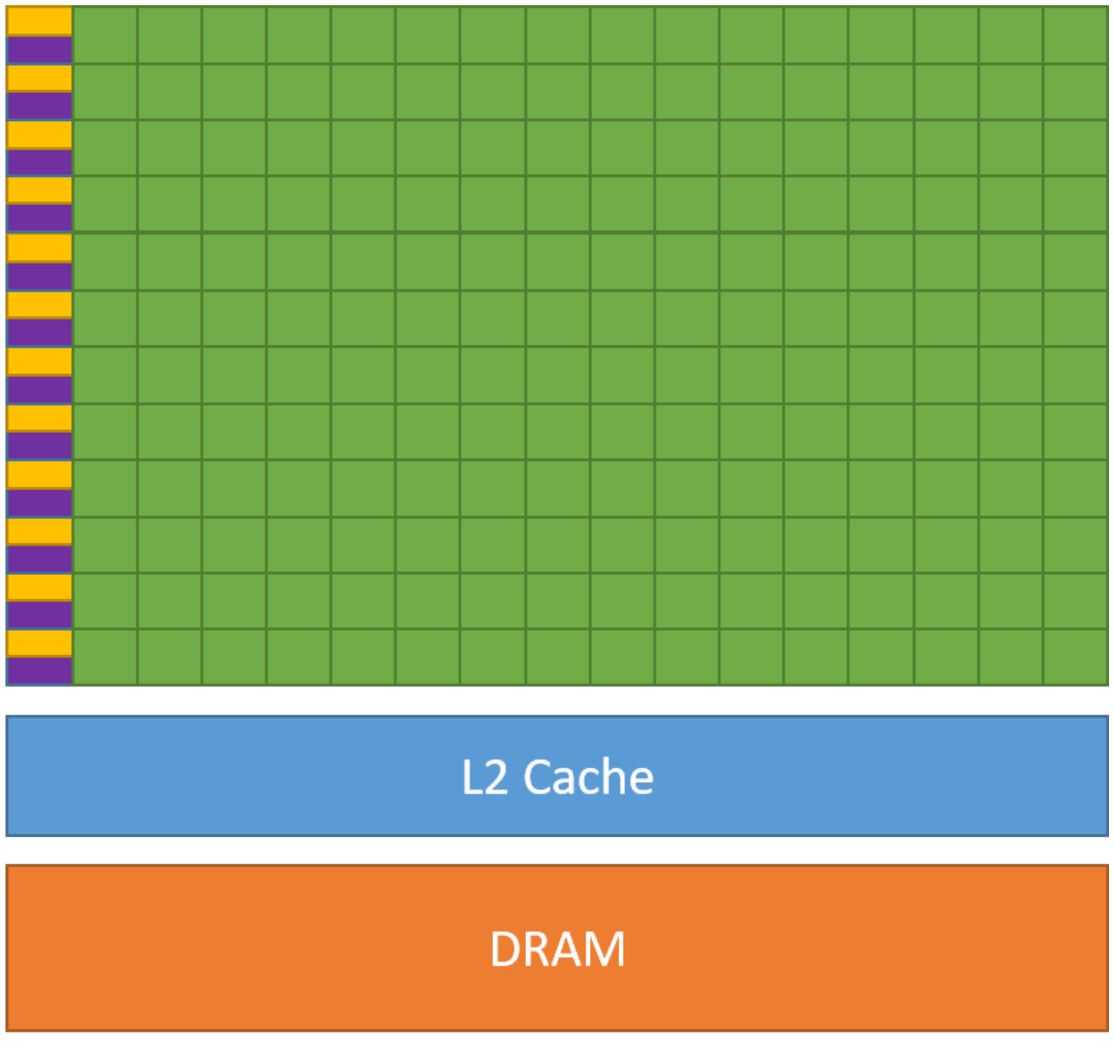
HPC Compute Node Schematic - Anatomy of a Workhorse



CPU vs GPU

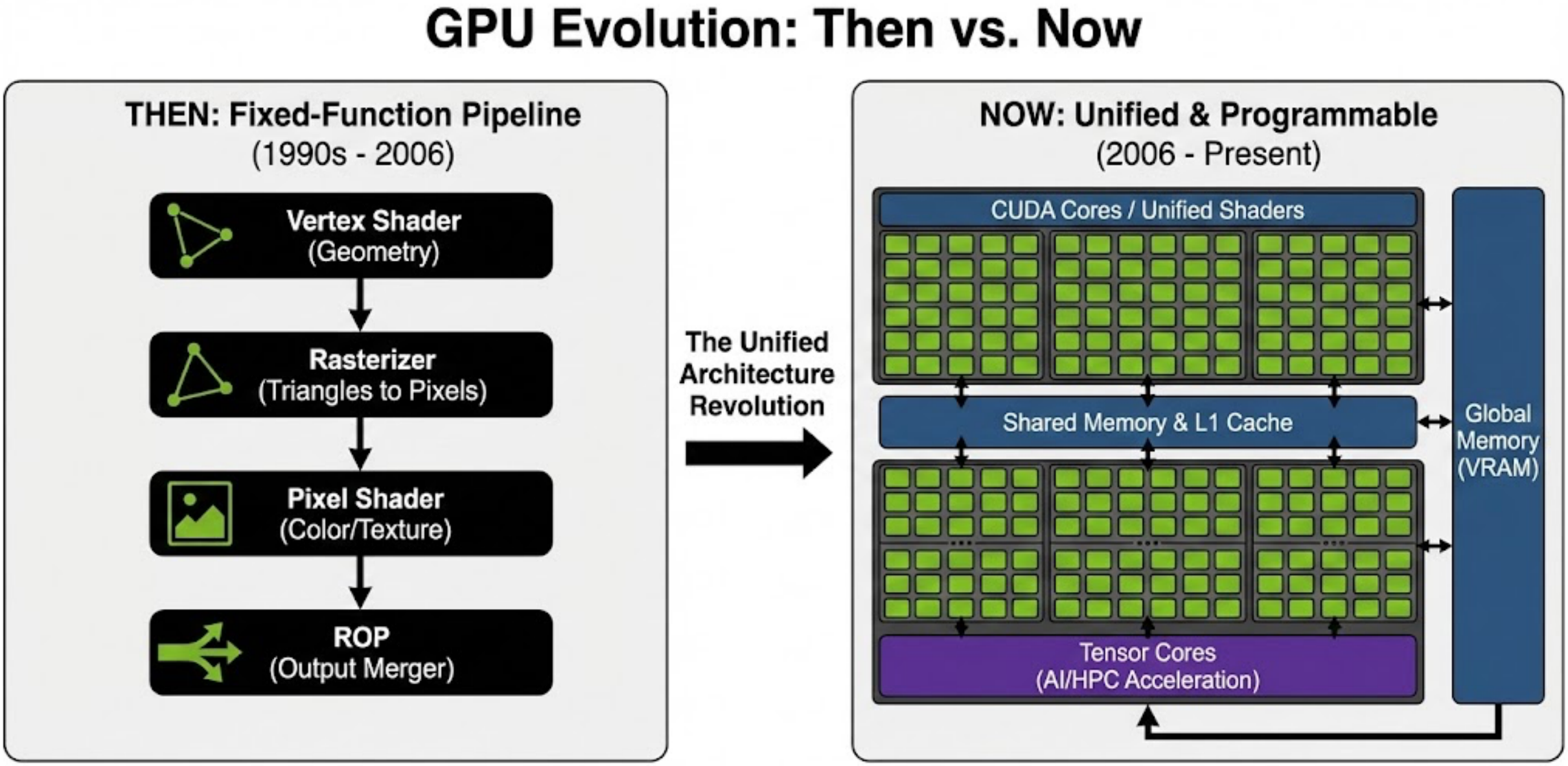


CPU

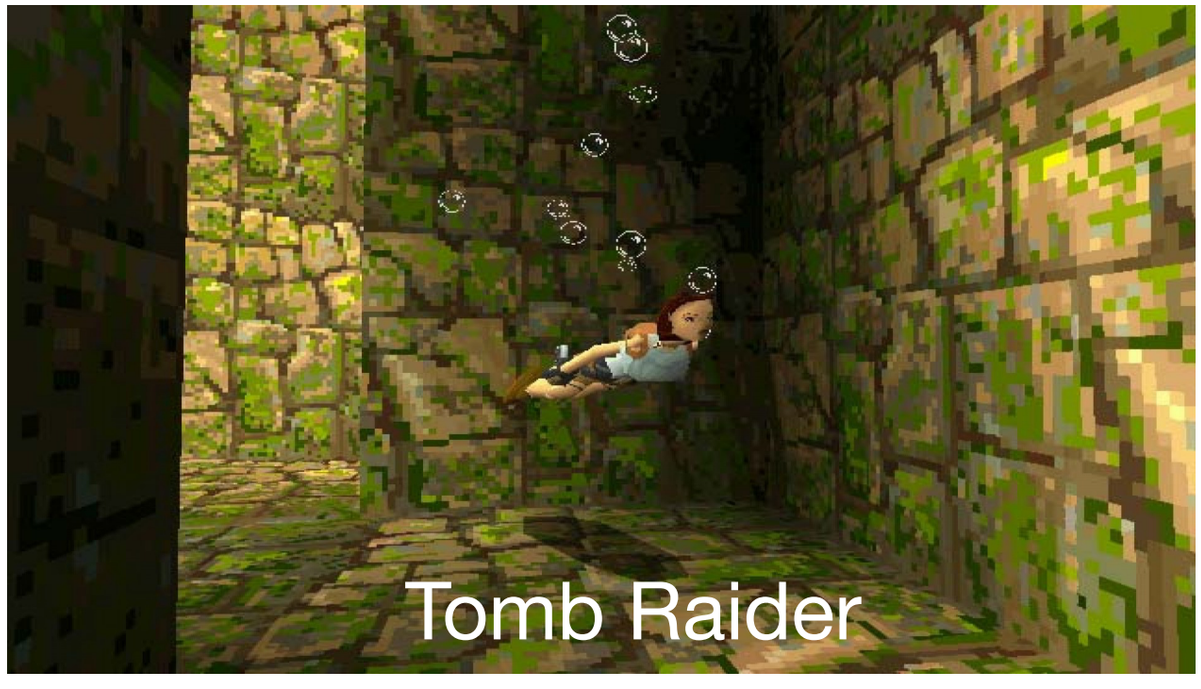


GPU

GPUs over the years



In the beginning…



In 2000, the growing NVIDIA bought and absorbed 3DFX

The birth of CUDA (2006)

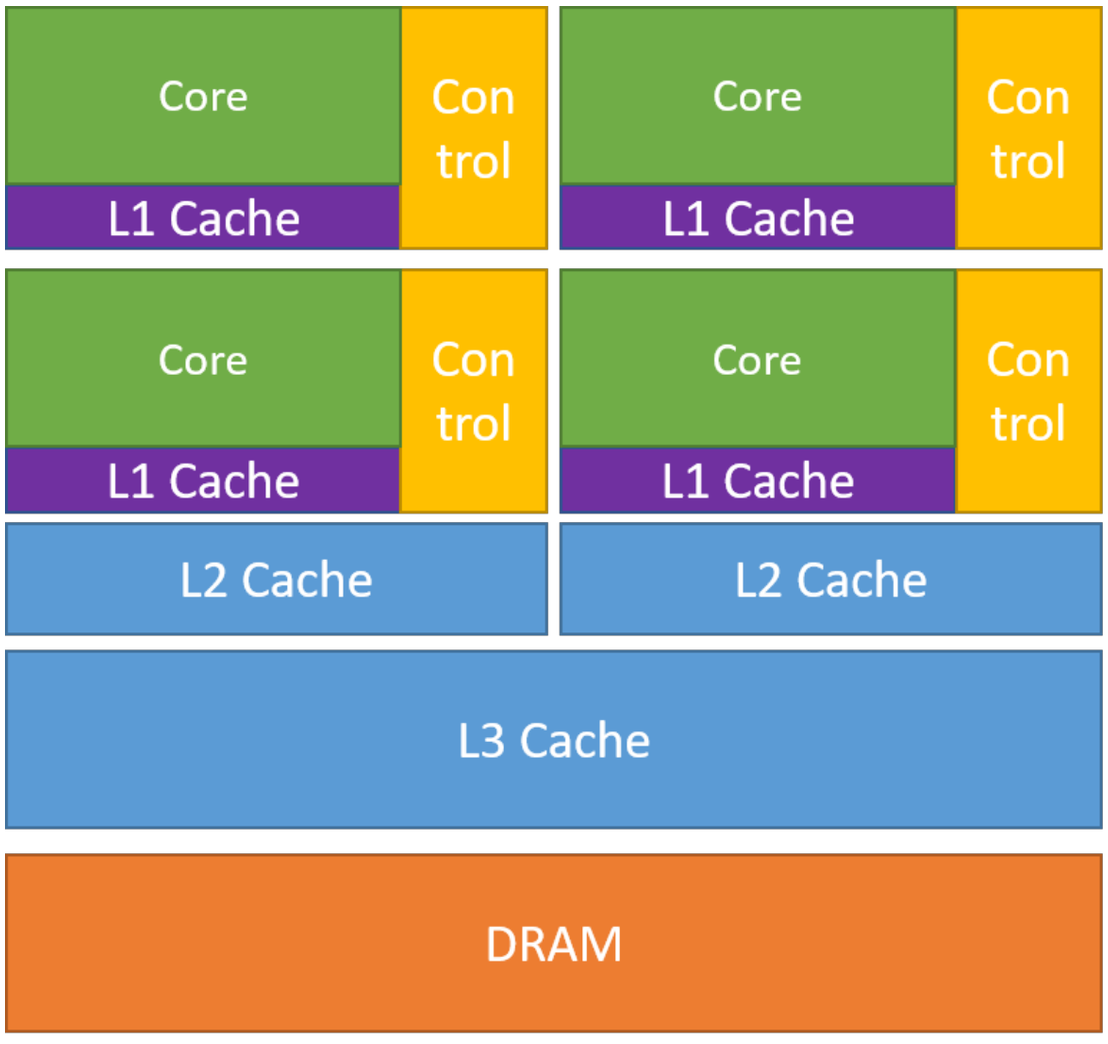
1. **Moving beyond videogames** - Transformed GPUs from specialized 3D rendering engines into general-purpose parallel processors.

2. **Emergence of CUDA language** - NVIDIA released a C-based language (**C**ompute **U**nified **D**evice **A**rchitecture) that allowed programmers to ignore graphics entirely

3. **Evolution of precision** - Early GPUs only cared about float32 (gaming quality). Float64 and float16 were added as it entered the realm of scientific computing and AI.

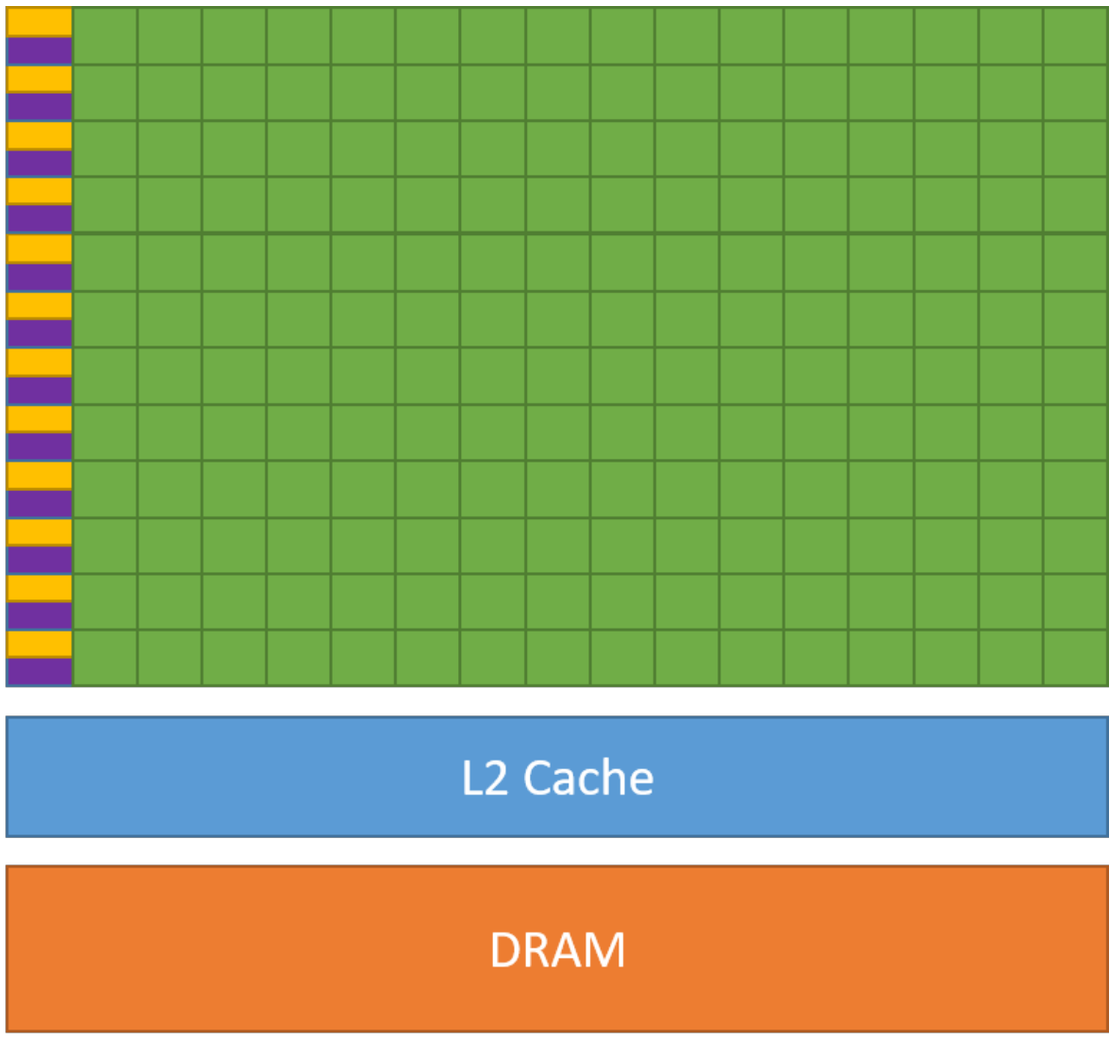
Host vs. Device

Host



CPU

Device



GPU

Requesting GPUs

Request an interactive job with a GPU on the DCC

```
nc153 $ srun --gres=gpu:1 -p gpu-common --pty bash
```

Checking for the available GPU

```
nc153 $ nvidia-smi
Wed Feb 18 21:03:12 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 575.57.08              Driver Version: 575.57.08      CUDA Version: 12.9     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 2080 Ti     On  |   00000000:04:00.0 Off |                  N/A |
| 30%   26C    P8             23W /  250W |       1MiB /  11264MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                      GPU Memory |
|        ID   ID                                                             Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Numba CUDA hello world

```python
import numpy as np
from numba import cuda

# 1. The Kernel (The GPU 'Worker')
@cuda.jit
def hello_cuda_kernel(io_array):
    # Determine the unique thread position in the 1D grid
    pos = cuda.grid(1)

    # Boundary check: Ensure we don't access memory outside the array
    if pos < io_array.size:
        # Every thread performs this math simultaneously on its own element
        io_array[pos] *= 2

# 2. Host Setup
data = np.arange(10**5, dtype=np.float64)
print(f"Original array on Host: {data}")

# 3. Step A: Move data to the Device (GPU)
# This is like sending a message in MPI
d_data = cuda.to_device(data)

# 4. Step B: Configure the Grid
# We'll use 5 blocks with 32 threads
threads_per_block = 32
blocks_per_grid = 5

# Launch the kernel
hello_cuda_kernel[blocks_per_grid, threads_per_block](d_data)

# 5. Step C: Move data back to the Host (CPU)
# This is like receiving a message in MPI
result = d_data.copy_to_host()

print(f"Result from Device:     {result}")
```

Transferring memory from host memory (RAM) to device memory (VRAM)

```python
import numpy as np
from numba import cuda

# Create a standard NumPy array (on the Host)
h_data = np.linspace(0, 100, 1000)

# Move it to the GPU
d_data = cuda.to_device(h_data)

# Don't waste time copying zeros from CPU to GPU
d_result = cuda.device_array((1,), dtype=np.float64)

print(type(h_data)) # <class 'numpy.ndarray'>
print(type(d_data)) # <class 'numba.cuda.cudadrv.devicearray.DeviceNDArray'>
print(type(d_result)) # <class 'numba.cuda.cudadrv.devicearray.DeviceNDArray'>
```

```
(cee690) nc153 $ python gpu_to_device.py
<class 'numpy.ndarray'>
<class 'numba.cuda.cudadrv.devicearray.DeviceNDArray'>
<class 'numba.cuda.cudadrv.devicearray.DeviceNDArray'>
```

Warning: The overhead due to transferring memory from RAM to VRAM can undercut the reason for using a GPU.

Configuring the “workers” and “teams”

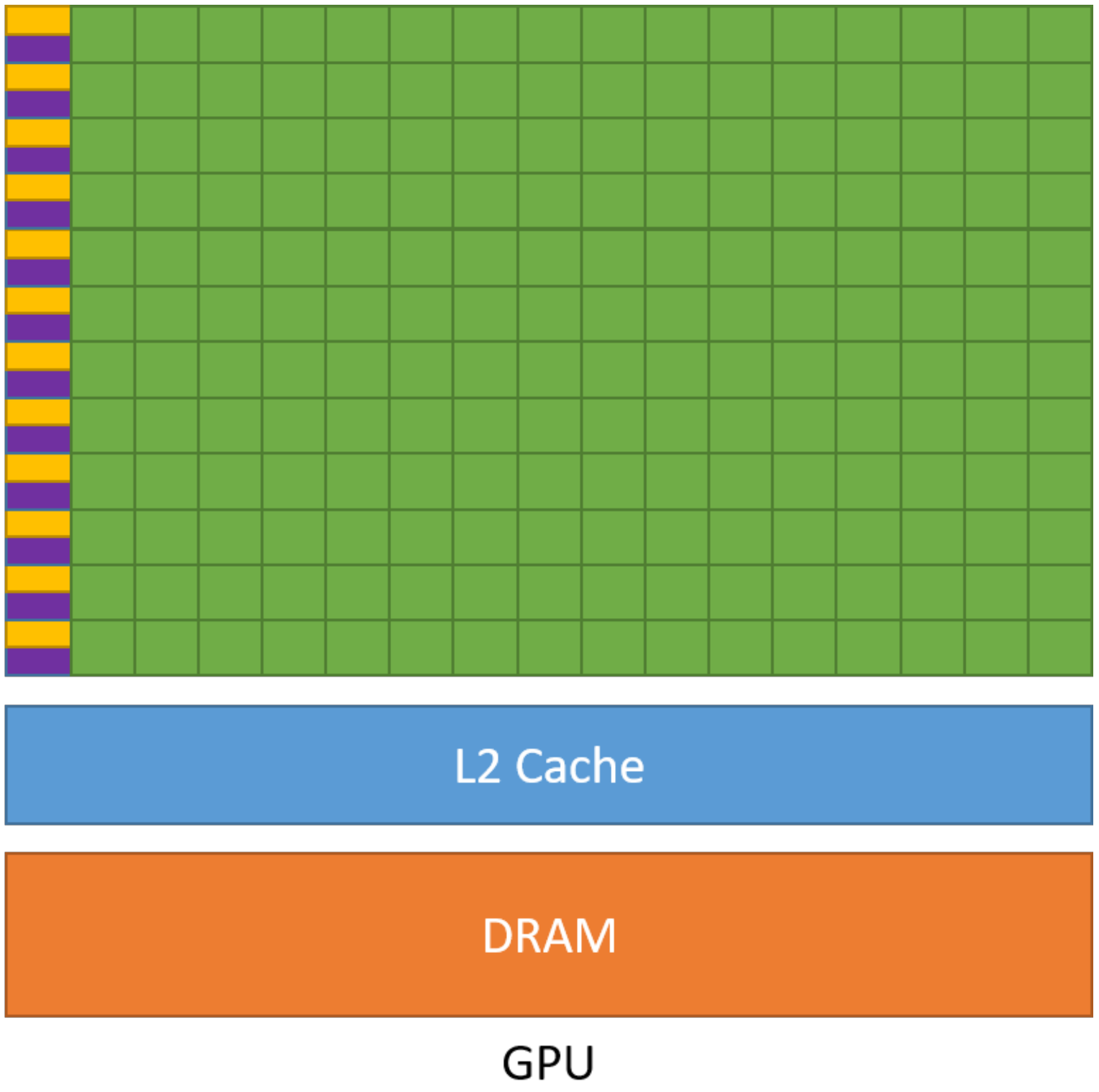
1. **The threads (“the worker”)** - The smallest unit. Each thread executes the kernel code.

2. **The block (“the office”)** - A group of threads that work together. They have shared memory that they can use to synchronize amongst themselves.

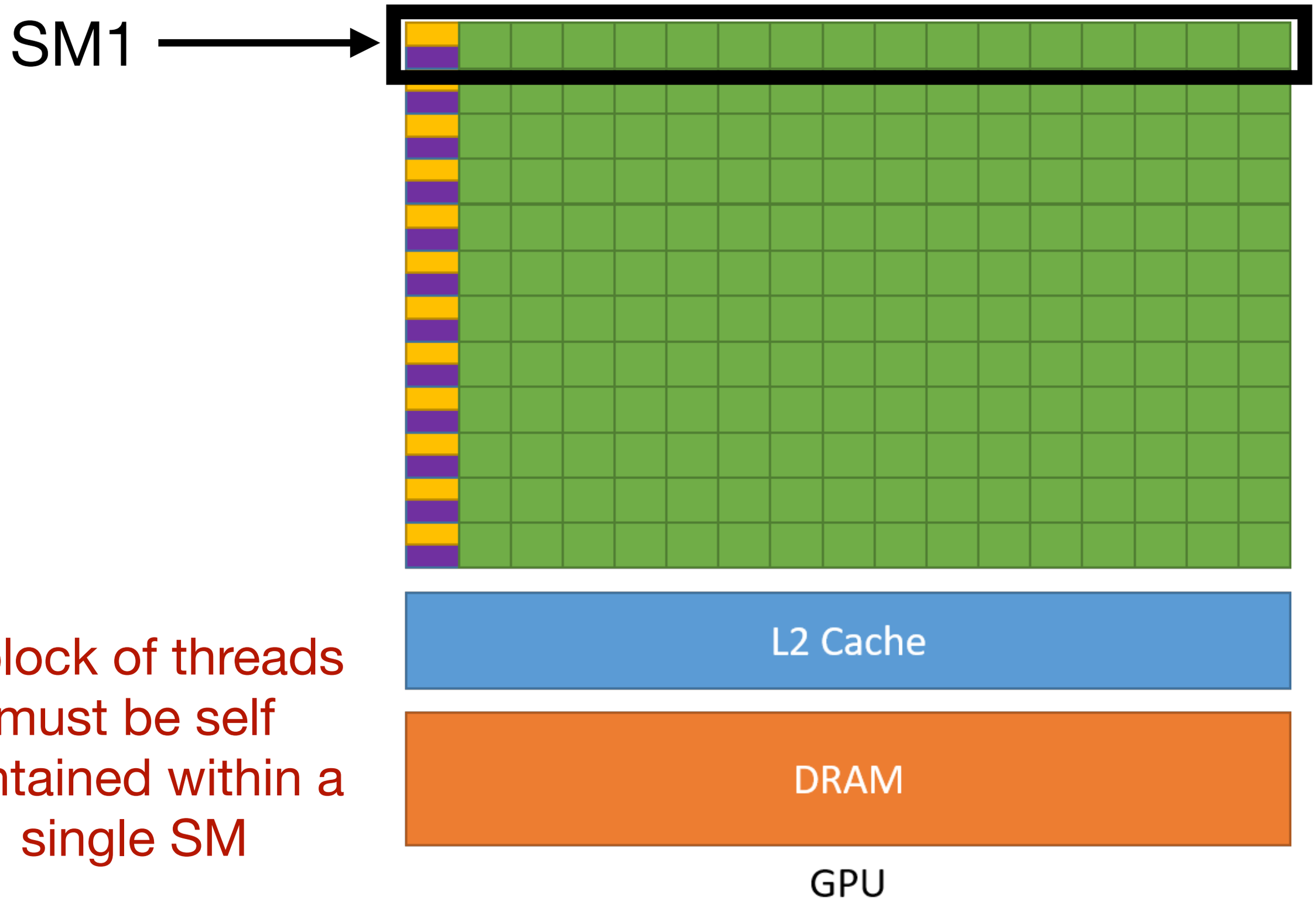
3. **The grid (“the building”)** - The collection of all blocks that will be used to complete the job.

Detour: Streaming multiprocessor (SM)

A block of threads must be self contained within a single SM

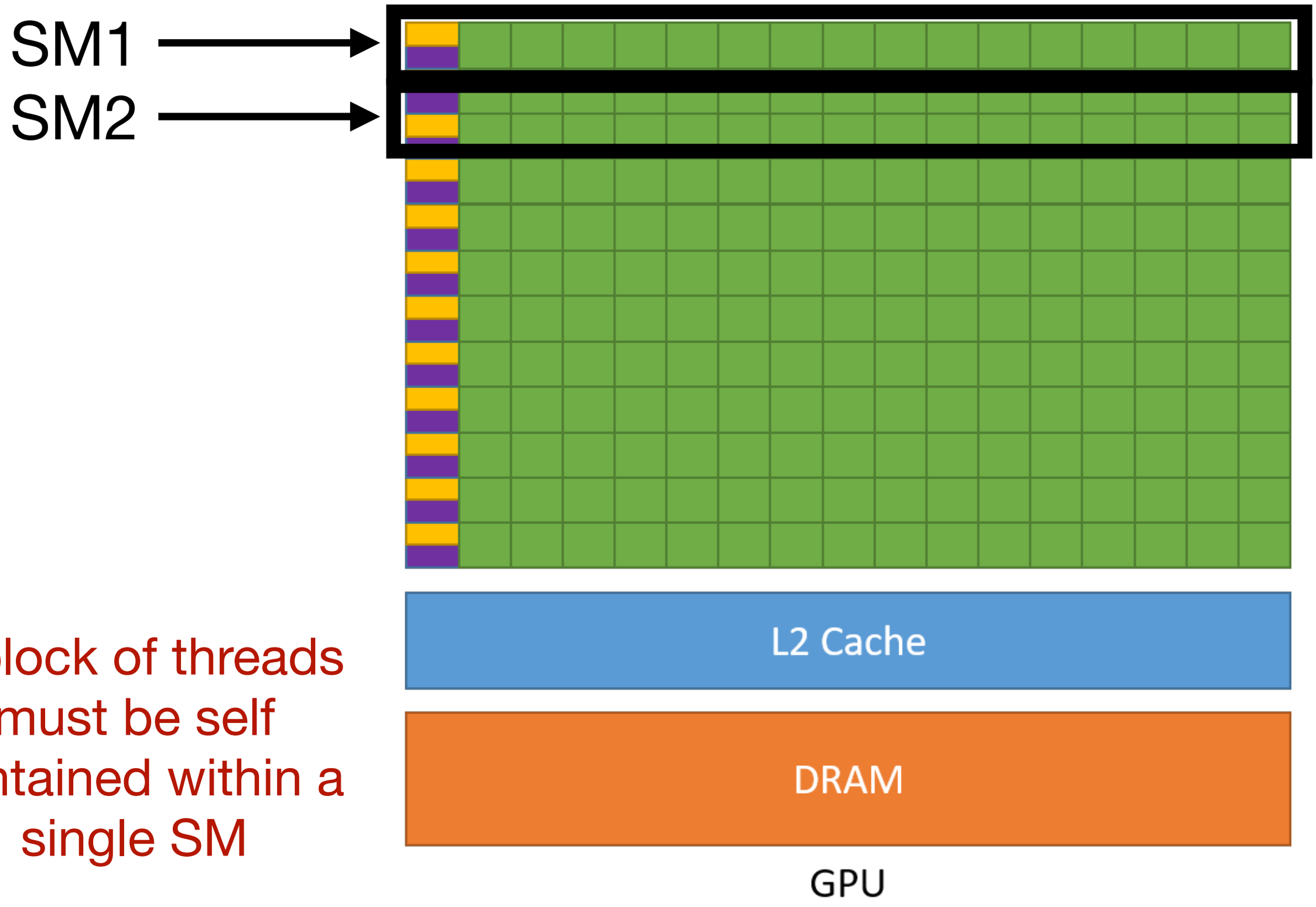


Detour: Streaming multiprocessor (SM)



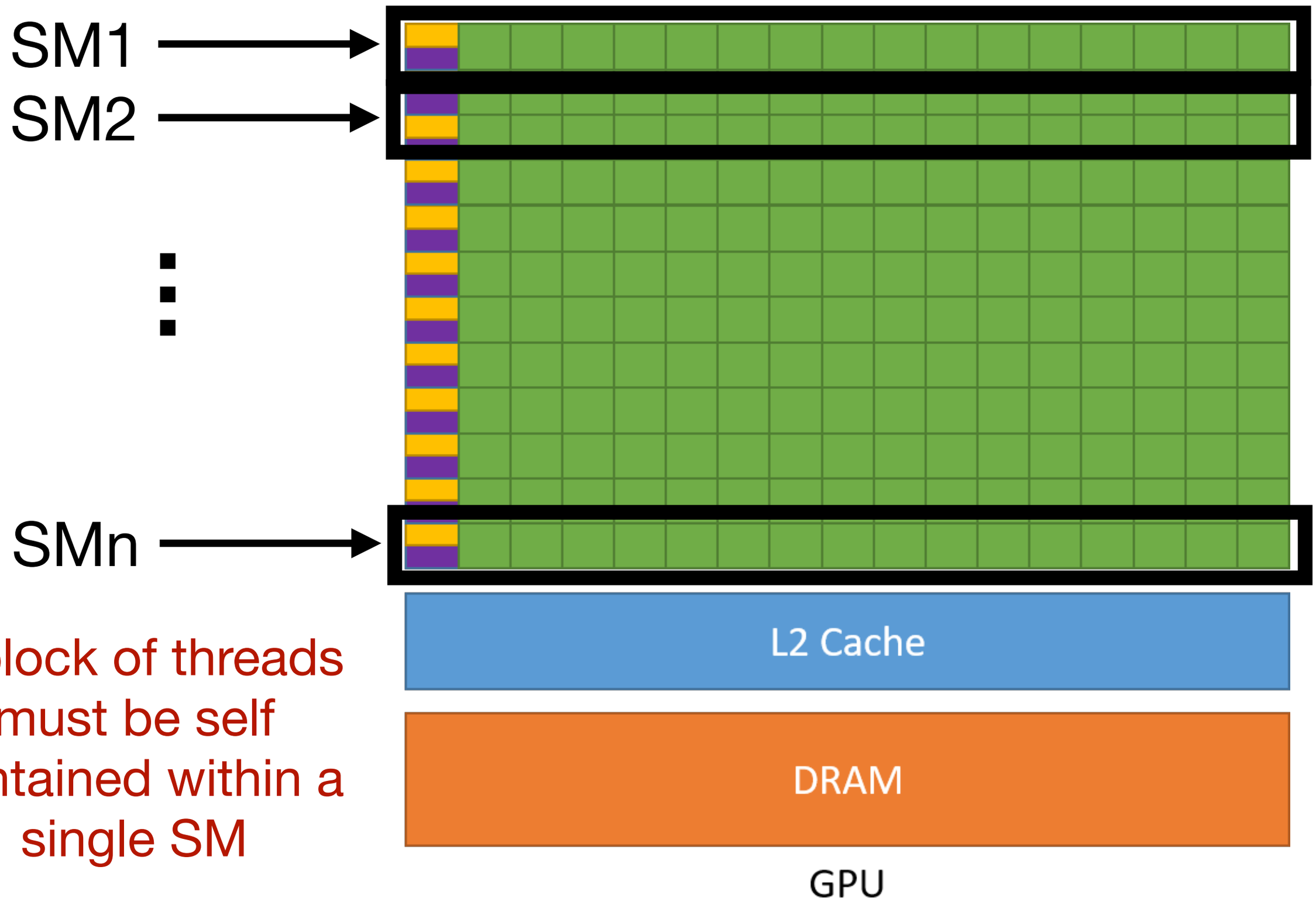
A block of threads must be self contained within a single SM

Detour: Streaming multiprocessor (SM)



A block of threads must be self contained within a single SM

Detour: Streaming multiprocessor (SM)



A block of threads must be self contained within a single SM

So what does my GPU actually look like?

```python
from numba import cuda

device = cuda.get_current_device()

# Use the .attributes dictionary for hardware-specific constants
# These keys are specifically defined in Numba's CUDA driver
num_sms = device.MULTIPROCESSOR_COUNT
max_threads_per_sm = device.MAX_THREADS_PER_MULTI_PROCESSOR
max_threads_per_block = device.MAX_THREADS_PER_BLOCK

print(f"Device Name: {device.name.decode('utf-8')}")
print(f"Number of SMs (Multiprocessors): {num_sms}")
print(f"Max Threads per SM:              {max_threads_per_sm}")
print(f"Max Threads per Block:           {max_threads_per_block}")
```

```
(cee690) nc153 $ python gpu_query_device.py
Device Name: NVIDIA GeForce RTX 2080 Ti
Number of SMs (Multiprocessors): 68
Max Threads per SM:              1024
Max Threads per Block:           1024
```

Max number of threads = 68*1024 = 65,536

Defining the threads and blocks

```
# 3. Step A: Move data to the Device (GPU)
# This is like sending a message in MPI
d_data = cuda.to_device(data)

# 4. Step B: Configure the Grid
# We'll use 1 block with 8 threads (since our array is tiny)
threads_per_block = 8
blocks_per_grid = 1
```

1. **The rule** - The number of threads per block will be a multiple of 32 (even if you defined it otherwise). Each group of 32 is called a “warp”. They always work at the same time.

2. **The standard**- Numbers of threads per block tend to be 128, 256, and 512.

3. **The limit** - Each GPU will have a maximum number of threads per block that be be used.

The kernel

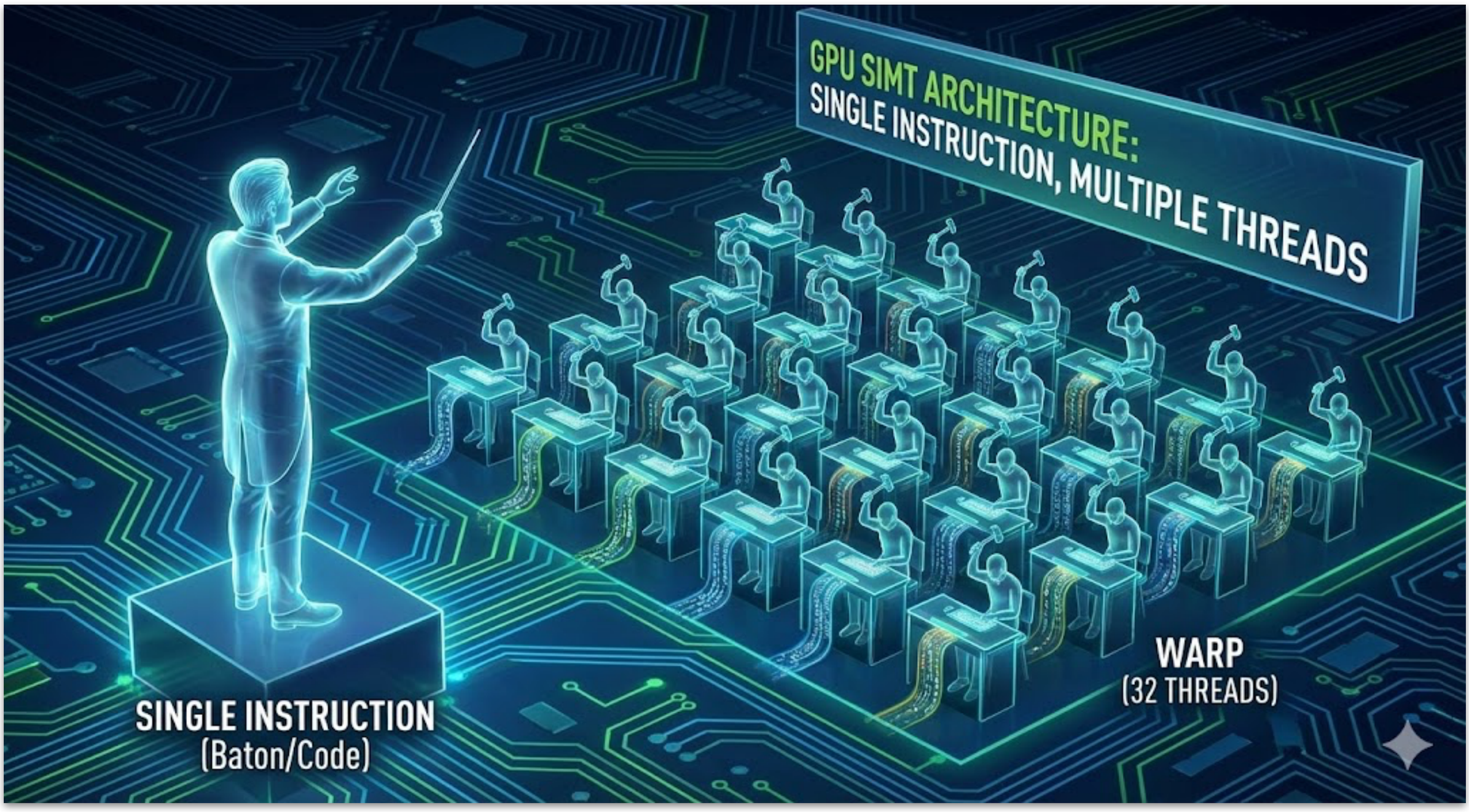
@cuda.jit compiles the code to PTX which is a low-level instruction set specific to NVIDIA GPUs

```python
# 1. The Kernel (The GPU 'Worker')
@cuda.jit
def hello_cuda_kernel(io_array):
    # Determine the unique thread position in the 1D grid
    rank = cuda.grid(1)
    stride = cuda.gridsize(1) # Total threads in the grid

    # Iterate through all the elements
    for i in range(rank,io_array.size,stride):
        io_array[i] *= 2
```

What goes inside the kernel needs to be very basic and generally non-pythonic (don’t initialize numpy arrays).

These threads are not very smart



Which thread am I?

```python
# 1. The Kernel (The GPU 'Worker')
@cuda.jit
def hello_cuda_kernel(io_array):
    # Determine the unique thread position in the 1D grid
    rank = cuda.grid(1)
    stride = cuda.gridsize(1) # Total threads in the grid

    # Iterate through all the elements
    for i in range(rank,io_array.size,stride):
        io_array[i] *= 2
```

How many threads are there?

```python
# 1. The Kernel (The GPU 'Worker')
@cuda.jit
def hello_cuda_kernel(io_array):
    # Determine the unique thread position in the 1D grid
    rank = cuda.grid(1)
    stride = cuda.gridsize(1) # Total threads in the grid

    # Iterate through all the elements
    for i in range(rank,io_array.size,stride):
        io_array[i] *= 2
```

“Submitting” the job to the GPU

```
# 4. Step B: Configure the Grid
# We'll use 5 blocks with 32 threads
threads_per_block = 32
blocks_per_grid = 5

# Launch the kernel
hello_cuda_kernel[blocks_per_grid, threads_per_block](d_data)
```

The square brackets are particular to CUDA in Python. It configures the “grid” (blocks/threads) on the GPU that will be used to run the kernel.

Back to the CPU

```
# 5. Step C: Move data back to the Host (CPU)
# This is like receiving a message in MPI
result = d_data.copy_to_host()

print(f"Result from Device:     {result}")
```

Copy the data on the GPU VRAM back to the RAM being used by the CPU. The CPU then prints the result.

“Calculate mean” revisited

```python
import time
import numpy as np
from numba import cuda, float64
import warnings
from numba.core.errors import NumbaPerformanceWarning

# --- 1. THE KERNEL (The "Parallel Region") ---
# This runs on the GPU. Think of this as the body of your 'parallel for'.
@cuda.jit
def mean_kernel_2d(data, global_result):
    rows, cols = data.shape

    # IDENTIFY: Who am I? (get 2D coordinates)
    start_x, start_y = cuda.grid(2)

    # STRIDE: How big is the army of threads?
    stride_x, stride_y = cuda.gridsize(2)

    # PRIVATE VARIABLES: Just like private(j) in OpenMP
    # Each thread keeps its own running total in a register (fastest memory)
    thread_sum = 0.0
    thread_count = 0

    # WORK: Grid-Stride Loop
    # Instead of "for i in range(rows)", we jump by the grid size.
    # This ensures every element i
    for i in range(start_x, rows,
        for j in range(start_y, co
```

```
(cee690) nc153 $ python gpu_calculate_mean.py
Running Numba CUDA Mean Calculation...
Result:     0.00012655884641267238
Time:       0.1255s
```

Libraries that you should know about

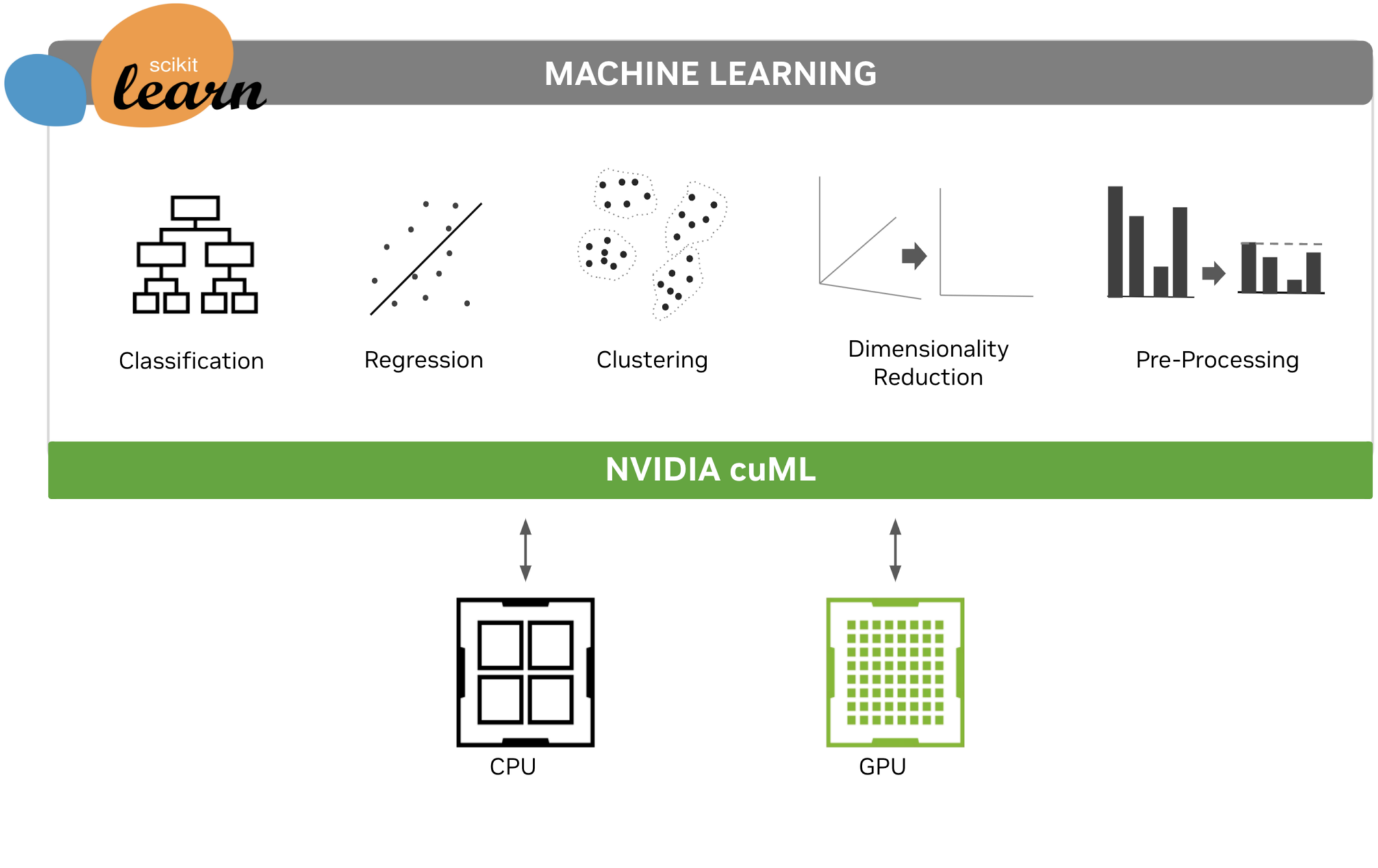
Drop-in replacement for Numpy but for the GPU

CuPy example

```python
# Standard CPU approach
import numpy as np
x_cpu = np.array([1, 2, 3])
l2_cpu = np.linalg.norm(x_cpu)

# Accelerated GPU approach
import cupy as cp
x_gpu = cp.array([1, 2, 3])
l2_gpu = cp.linalg.norm(x_gpu)
```

CuML


scikit learn
MACHINE LEARNING
Classification
Regression
Clustering
Dimensionality Reduction
Pre-Processing
NVIDIA cuML
CPU
GPU

PyCUDA: Lowest-level/most advanced use of CUDA in Python

```
# 1. The Raw C++ Kernel (as a Python string)
mod = SourceModule("""
__global__ void multiply_them(float *dest, float *a, float *b)
{
  const int i = threadIdx.x + blockIdx.x * blockDim.x;
  dest[i] = a[i] * b[i];
}
""")

multiply_them = mod.get_function("multiply_them")

# 2. Data setup
a = np.random.randn(400).astype(np.float32)
b = np.random.randn(400).astype(np.float32)

dest = np.zeros_like(a)

# 3. Launching (PyCUDA handles the data movement automatically here)
multiply_them(
        drv.Out(dest), drv.In(a), drv.In(b),
        block=(400,1,1), grid=(1,1))
```